

Assignment 13

Dynamic String Buffer in C

ARITRA TALUKDAR;CSB24034

Aim

To implement a Dynamic String Buffer in C that automatically grows using dynamic memory allocation and prevents buffer overflows and memory leaks.

Objectives

- To understand dynamic memory allocation in C.
- To use `malloc()` and `realloc()` safely.
- To implement automatic buffer resizing.
- To prevent memory leaks using proper memory deallocation.

Theory

In C programming, fixed-size character arrays can cause buffer overflow problems when large strings are stored. Dynamic memory allocation allows memory to grow during program execution.

The program uses:

- `malloc()` for initial memory allocation
- `realloc()` for resizing the buffer dynamically
- `free()` to release allocated memory

A structure named `StringBuffer` stores:

- Pointer to character data
- Current length of the string
- Total buffer capacity

Whenever the buffer becomes full, its capacity is doubled automatically.

Algorithm

1. Create a structure containing data pointer, length, and capacity.
2. Allocate initial memory using `malloc()`.
3. Take user input strings.
4. Before appending:
 - Check whether capacity is sufficient.
 - If not, double the capacity using `realloc()`.
5. Append the string to the buffer.
6. Display updated string and buffer details.
7. Free all allocated memory before program termination.

Program

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  typedef struct {
6      char *data;
7      size_t length;
8      size_t capacity;
9  } StringBuffer;
10
11  /* Initialize StringBuffer */
12  StringBuffer* sb_init(size_t initial_capacity) {
13
14      StringBuffer *sb = (StringBuffer *)malloc(sizeof(StringBuffer));
15
16      if (sb == NULL) {
17          printf("Memory allocation failed.\n");
18          return NULL;
19      }
20
21      sb->data = (char *)malloc(initial_capacity);
22
23      if (sb->data == NULL) {
24          free(sb);
25          return NULL;
26      }
27
28      sb->length = 0;
29      sb->capacity = initial_capacity;
30      sb->data[0] = '\0';
```

```

31
32     return sb;
33 }
34
35 /* Append string */
36 void sb_append(StringBuffer *sb, const char *str) {
37
38     size_t str_len = strlen(str);
39
40     while (sb->length + str_len + 1 > sb->capacity) {
41
42         size_t new_capacity = sb->capacity * 2;
43
44         char *temp = (char *)realloc(sb->data, new_capacity);
45
46         if (temp == NULL) {
47             printf("Reallocation failed.\n");
48             return;
49         }
50
51         sb->data = temp;
52         sb->capacity = new_capacity;
53
54         printf("Buffer resized to %zu\n", sb->capacity);
55     }
56
57     strcat(sb->data, str);
58     sb->length += str_len;
59 }
60
61 /* Free memory */
62 void sb_free(StringBuffer *sb) {
63
64     if (sb != NULL) {
65         free(sb->data);
66         free(sb);
67     }
68 }
69
70 int main() {
71
72     size_t initial_capacity;
73     int choice;
74     char input[200];
75
76     printf("Enter initial capacity: ");
77     scanf("%zu", &initial_capacity);
78
79     getchar();
80
81     StringBuffer *sb = sb_init(initial_capacity);

```

```

82
83     if (sb == NULL) {
84         return 1;
85     }
86
87     do {
88
89         printf("\n1. Append String\n");
90         printf("2. Display String\n");
91         printf("3. Display Details\n");
92         printf("4. Exit\n");
93
94         printf("Enter choice: ");
95         scanf("%d", &choice);
96
97         getchar();
98
99         switch(choice) {
100
101             case 1:
102
103                 printf("Enter string: ");
104                 fgets(input, sizeof(input), stdin);
105
106                 input[strcspn(input, "\n")] = '\0';
107
108                 sb_append(sb, input);
109
110                 break;
111
112             case 2:
113
114                 printf("Current String: %s\n", sb->data);
115
116                 break;
117
118             case 3:
119
120                 printf("Length = %zu\n", sb->length);
121                 printf("Capacity = %zu\n", sb->capacity);
122
123                 break;
124
125             case 4:
126
127                 sb_free(sb);
128                 printf("Memory freed.\n");
129
130                 break;
131
132             default:

```

```

133
134         printf("Invalid choice.\n");
135     }
136
137     } while(choice != 4);
138
139     return 0;
140 }

```

Sample Output

Enter initial capacity: 8

1. Append String
2. Display String
3. Display Details
4. Exit

Enter choice: 1

Enter string: Hello

Enter choice: 1

Enter string: Dynamic Buffer

Buffer resized to 16

Buffer resized to 32

Enter choice: 2

Current String: HelloDynamic Buffer

Enter choice: 3

Length = 20

Capacity = 32

Conclusion

The Dynamic String Buffer was successfully implemented using dynamic memory allocation in C. The buffer automatically resized itself whenever required, preventing buffer overflows. Proper use of `malloc()`, `realloc()`, and `free()` ensured safe memory management and avoided memory leaks.